

UNIVERSIDADE DE SÃO PAULO  
INSTITUTO DE CIÊNCIAS MATEMÁTICAS E DE COMPUTAÇÃO



**Relatório Projeto Java Event Planner**

*SSC0504 - Programação Orientada a Objetos (2026)*

**Grupo 12**

João Vitor Salvini

11218664

Jaqueline Paes de Almeida

8927485

Lucas Jacinto Mariano

11879922

São Carlos

2026

## 1. Requisitos

O projeto escolhido foi o Java Event Planner (versão compacta), uma aplicação desktop em Java com Swing para gerenciamento de eventos em calendário.

### 1.1 Requisitos Funcionais

**RF01 — Visualização do Calendário Mensal:** O sistema apresenta uma janela principal dividida de forma clara, contendo a grade do calendário mensal de um lado e a listagem da agenda do dia selecionado do outro.

**RF02 — Seleção de Datas:** Ao clicar em um botão de data na grade do calendário, o sistema atualiza dinamicamente a barra lateral e exibe apenas os eventos cadastrados para o dia em questão.

**RF03 — Destaque Visual de Compromissos:** As datas que possuem pelo menos um evento cadastrado aparecem destacadas na grade através da alteração de sua cor de fundo para azul claro e pela adição de um marcador de texto puro (\*) ao lado do número do dia.

**RF04 — Gerenciamento de Eventos (CRUD):** O sistema permite criar, visualizar, editar e excluir eventos contendo os atributos: título, data, horário (com suporte a eventos de dia inteiro), local, descrição e categoria.

**RF05 — Lembretes de Inicialização:** Durante a inicialização, o sistema realiza uma varredura automática nos dados e exibe um alerta em formato *pop-up* caso existam eventos cujos dias de antecedência configurados coincidam com a data atual (janela das próximas 24 horas).

**RF06 — Salvamento Automático de Dados:** O sistema persiste todas as modificações (adições, edições e exclusões) imediatamente em um arquivo de texto local estipulado (events\_data.txt).

**RF07 — Carregamento Automático de Dados:** Ao iniciar a aplicação, o sistema lê o arquivo de dados local e reconstrói a lista de eventos na memória RAM de forma transparente ao usuário.

### 1.2 Requisitos de Validação e Exceções

**RV01 — Validação dos Dados do Evento:** O sistema impede de forma estrita o cadastro ou a atualização de eventos que possuam o campo de título vazio ou formatos textuais inválidos para os campos de data e horário.

**RV02 — Tratamento de Arquivo Ausente:** Caso o arquivo de dados local ainda não exista (como na primeira execução da aplicação), o sistema cria o ambiente e carrega a interface vazia de forma silenciosa e amigável, sem lançar erros ou travar.

**RV03 — Tolerância a Dados Corrompidos:** Se o arquivo de persistência contém linhas com dados malformados ou corrompidos, o sistema ignora a linha defeituosa, emite um aviso interno no console de erros (`System.err`) e continua a leitura das linhas subsequentes sem interromper o funcionamento da aplicação.

**RV04 — Mensagens ao Usuário:** Toda falha de validação gerada por ações do usuário na interface gráfica é reportada por meio de caixas de diálogo explícitas (`JOptionPane`), omitindo detalhes técnicos internos ou logs de pilhas de erro (*stack traces*).

### 1.3 Requisitos Adicionais da Implementação

**RA01 — Navegação Entre Meses:** A barra superior do calendário dispõe de botões direcionais manuais (`< Prev` e `Next >`) para avançar e retroceder os meses de exibição da grade.

**RA02 — Botão "Today" (Hoje):** A interface possui um botão de atalho rápido que, ao ser acionado, redefine a exibição do calendário diretamente para o mês e ano correntes do sistema operacional.

**RA04 — Organização Estática da Interface:** A disposição das telas é controlada de forma limpa por gerenciadores de layout nativos e pela divisão física estipulada pelo componente `JSplitPane`.

### 1.4 Requisitos Não Funcionais

**RNF01 — Linguagem e Tecnologia:** O projeto é desenvolvido integralmente em Java nativo convencional, fazendo uso exclusivo da biblioteca `javax.swing` para a construção de sua interface desktop.

**RNF02 — Lógica de Programação Tradicional:** A busca, filtragem e iteração de dados ocorrem por meio de estruturas e laços de repetição tradicionais (`for-each` e `while`), evitando o uso de abstrações funcionais avançadas como a Stream API ou expressões Lambda.

**RNF03 — Estruturas de Memória Dinâmica:** A coleção de registros em tempo de execução fica centralizada em estruturas do tipo `ArrayList` manipuladas por uma classe controladora de persistência dedicada.

**RNF04 — Portabilidade e Organização Única:** Os arquivos de código fonte do projeto residem todos no mesmo nível de diretório (pasta raiz única), sem subdivisões de pacotes físicos, garantindo a compilação direta via terminal de comando por meio do utilitário padrão `javac *.java`.

## 2. Descrição do Projeto

O projeto implementa a versão compacta do **Java Event Planner**, um sistema de agendamento de eventos e calendário construído em Java com a biblioteca Swing. A arquitetura foi desenvolvida para atender a todos os requisitos funcionais solicitados na versão compacta.

A aplicação está dividida em duas áreas principais na interface: um calendário mensal interativo à esquerda e a agenda diária detalhada à direita.

### Implementação das Funcionalidades:

- **Gestão de Eventos (CRUD):** O usuário pode adicionar, editar e excluir eventos. As informações (título, data, hora, local, descrição, categoria e dias de antecedência para o lembrete) são coletadas através de uma janela modal (EventDialog) projetada para reaproveitamento de código tanto na criação quanto na edição. Ao criar um novo evento, o sistema herda automaticamente a data que estava selecionada na interface principal.
- **Navegação e Interface:** A classe EventPlannerGUI gerencia uma grade dinâmica usando GridLayout. A lógica de renderização calcula o dia da semana do início do mês e injeta espaços vazios (blanks) para que os dias fiquem alinhados corretamente (Domingo a Sábado). Dias com eventos recebem uma cor de fundo distinta e exibem a contagem de compromissos.
- **Lembretes de Inicialização:** Atendendo à restrição da versão compacta (sem uso de *threads* em background), a classe EventManager possui o método getUpcomingReminders(), que varre a lista de eventos assim que o programa é aberto, calculando a diferença de dias. Se houver eventos dentro da janela de antecedência definida pelo usuário, um alerta em *pop-up* é exibido.
- **Persistência de Dados:** O sistema utiliza BufferedReader e BufferedWriter para ler e salvar os dados em um arquivo de texto local (events\_data.txt), usando o caractere | como delimitador. A gravação ocorre automaticamente a cada alteração (adição, edição ou exclusão), garantindo que o usuário não perca dados caso feche a janela abruptamente.



### 3. Comentários sobre o código

O código foi estruturado com foco em **Programação Orientada a Objetos (POO)** e manutenção. Abaixo estão as principais decisões de design e boas práticas adotadas:

- **Encapsulamento e Validação Interna:** A classe `Event` mantém todos os seus campos privados. Além disso, a validação de regras de negócio (como garantir que o título do evento não seja vazio) foi encapsulada dentro da própria classe através do método `validateTitle()`, que é invocado pelos construtores e pelo método `setTitle()`.
- **Uso de Enumerações (Type Safety):** A utilização do enum `EventCategory` restringe a criação de eventos apenas às categorias permitidas (*Meeting, Birthday, Appointment, Other*), evitando erros de digitação por parte do usuário e facilitando a renderização visual na lista.
- **Tratamento Gracioso de Exceções:** \* *Falhas de I/O:* Se o arquivo de texto estiver ausente na primeira execução, o método `loadFromFile()` captura silenciosamente a situação e inicia uma lista vazia, em vez de gerar um `FileNotFoundException` que derrubaria o sistema.
  - *Erros de Formatação:* Ao ler o arquivo, o bloco `try-catch` trata `DateTimeParseException`. Se uma linha do arquivo estiver corrompida, o sistema apenas pula aquela linha (imprimindo um aviso no console) e continua carregando o resto da agenda.
- **Simplicidade e Performance no Swing:** Optou-se por remover as tags HTML que estilizavam os botões da grade do calendário nas iterações iniciais. A renderização visual agora depende estritamente das propriedades nativas do Swing (concatenação de strings e alteração via `setBackground()`). Essa decisão tornou a atualização da grade (`revalidate` e `repaint`) mais leve e deixou o código da classe `EventPlannerGUI` muito mais limpo e legível.

### 4. Plano de testes

Como a aplicação depende fortemente da interação visual do usuário, o plano de testes foca na execução manual de cenários baseados nos requisitos (*Black-box testing*), validando o fluxo da interface, a persistência e a resiliência contra falhas.

#### Cenário de Teste 1: Validação de Inputs e Exceções

- *Ação:* Clicar em "Add" e tentar salvar o formulário com o campo "Title" em branco.
- *Resultado Esperado:* O sistema não deve fechar. Um `JOptionPane` com ícone de erro deve alertar que o título não pode ser vazio.
- *Ação:* Digitar "Hoje" no campo "Date" e "10h" no campo "Time".

- *Resultado Esperado:* A conversão irá falhar. O sistema deve capturar a exceção e exibir um alerta amigável instruindo o uso do formato YYYY-MM-DD e HH:MM.

### Cenário de Teste 2: Lógica de Destaque no Calendário (Visual Highlight)

- *Ação:* Selecionar o dia 15 do mês atual e adicionar um evento válido.
- *Resultado Esperado:* Ao fechar o formulário, o botão correspondente ao dia 15 no calendário deve imediatamente mudar de cor e atualizar seu texto para incluir "(1\*)", indicando a presença de um evento.
- *Ação:* Excluir o evento recém-criado.
- *Resultado Esperado:* O dia 15 deve voltar instantaneamente à cor branca.

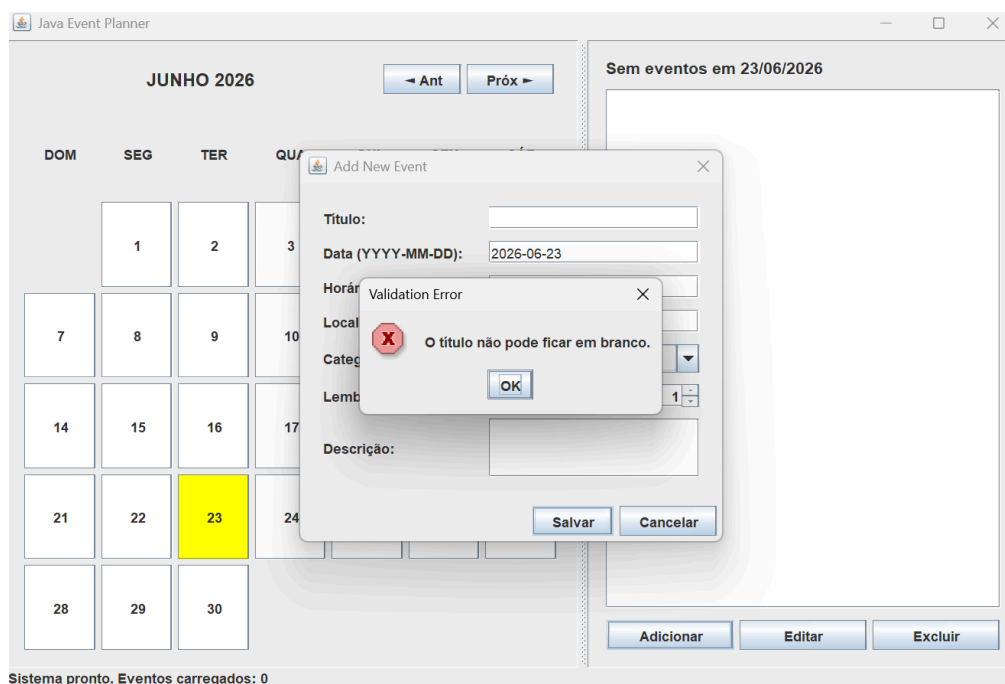
### Cenário de Teste 3: Disparo de Lembretes na Inicialização (Startup Reminders)

- *Ação:* Criar um evento para a data de amanhã, definindo a configuração "Reminder (Days Before)" para o valor 1.
- *Ação:* Fechar o programa e abri-lo novamente.
- *Resultado Esperado:* Antes que a janela principal seja liberada para uso, um JOptionPane informativo deve aparecer automaticamente listando o título do evento criado, validando a lógica matemática do método getUpcomingReminders().

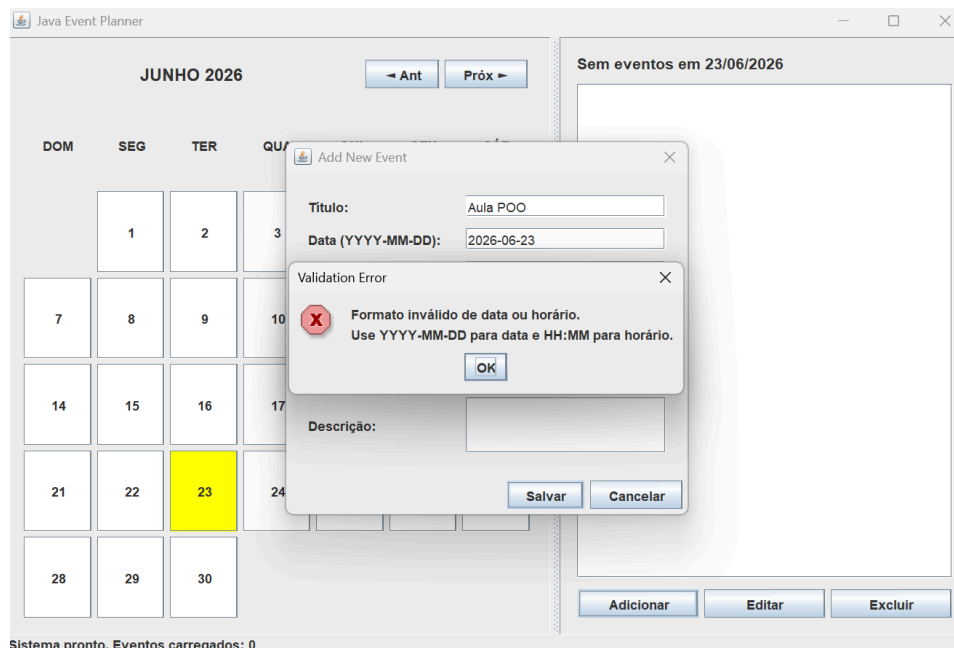
## 5. Resultados dos Testes

### 5.1. Cenário de Teste 1: Validação de Inputs e Exceções

Ao se tentar adicionar um evento com o título em branco, uma mensagem de erro foi exibida.

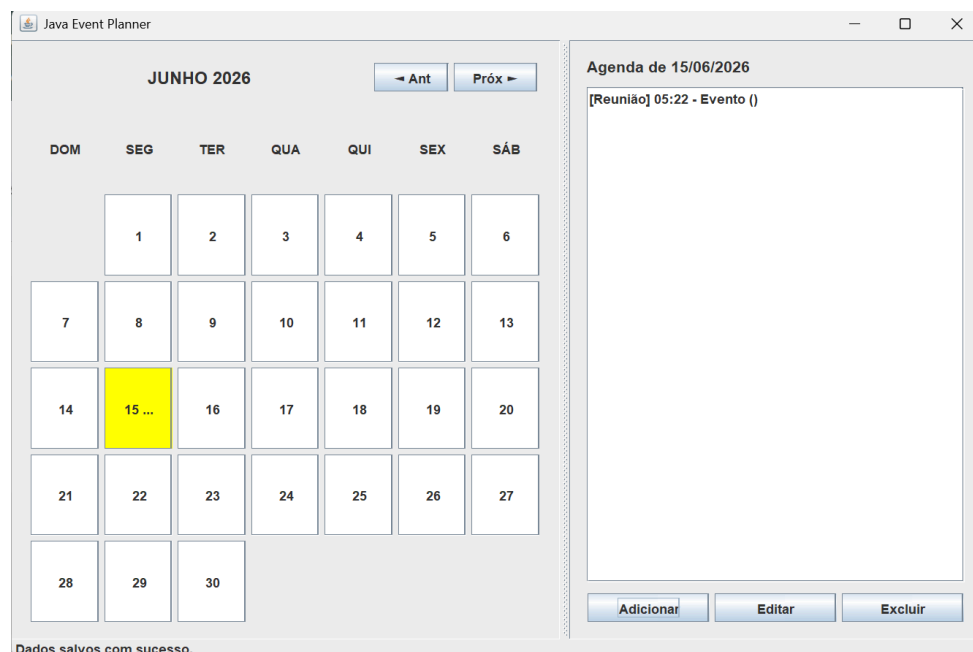


Ao se tentar adicionar um horário no formato inválido, uma mensagem de erro foi exibida, orientando o usuário quanto ao formato correto.

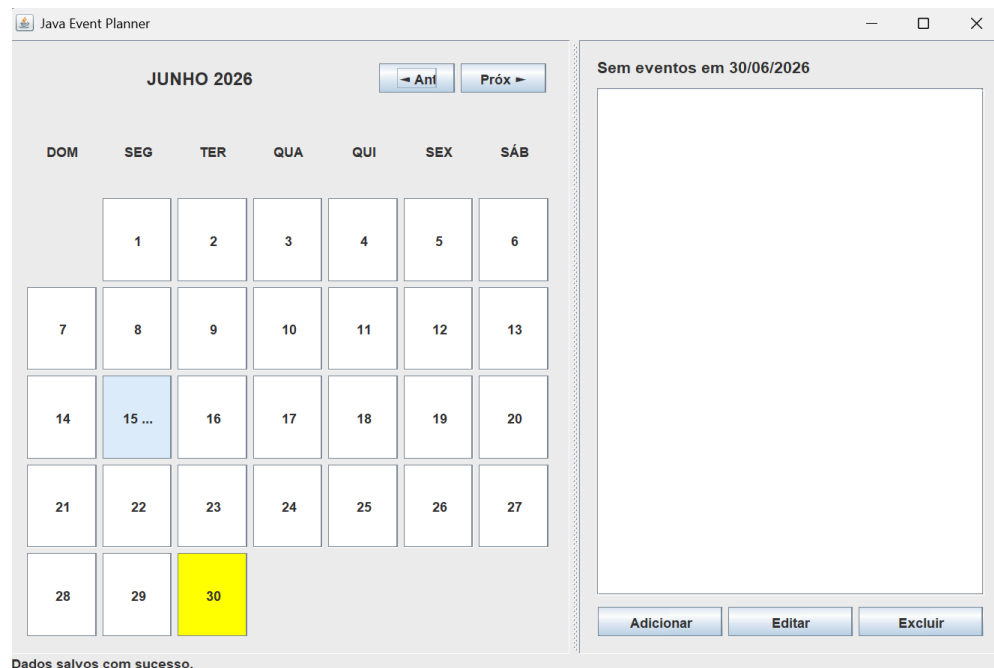


## 5.2. Cenário de Teste 2: Lógica de Destaque no Calendário (Visual Highlight)

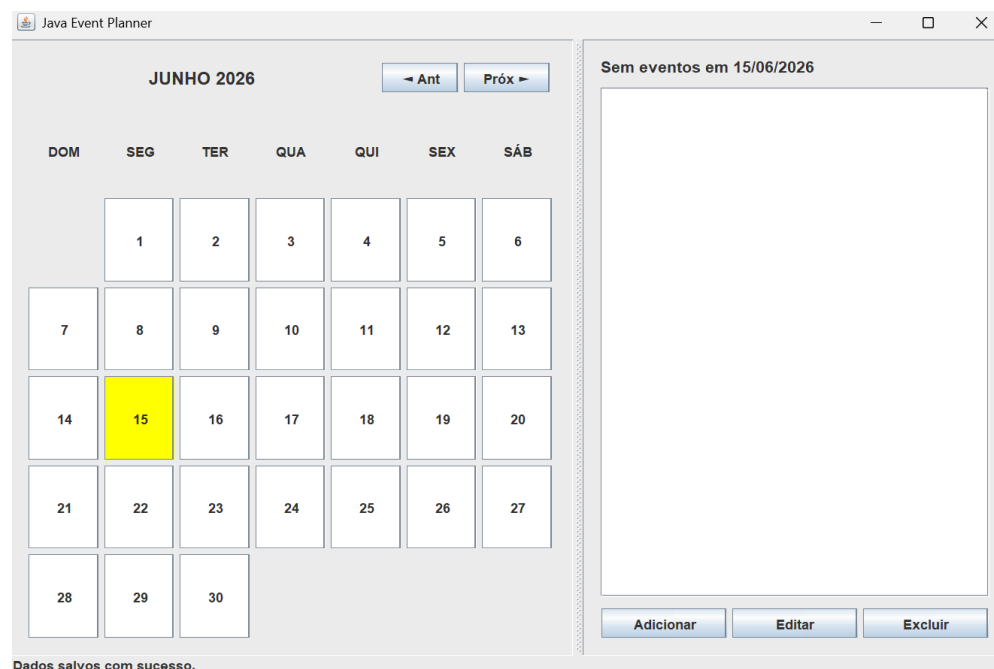
Foi criado um evento no dia 15 de Junho. Ao clicar neste dia, a janela da agenda do lado direito exibiu o evento cadastrado. Além disso, quando se clica em um dia qualquer, sua cor fica amarela.



Ao se clicar em um outro dia qualquer, é possível notar que o dia 15 ficou com um tom azulado, indicando que há um evento neste dia.



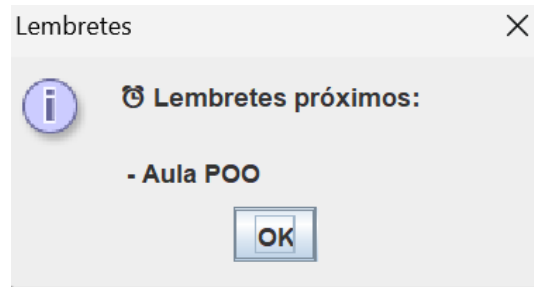
Após a remoção evento, o dia 15 voltou a ficar branco, indicando que não havia eventos agendados neste dia, como era esperado.





### 5.3. Cenário de Teste 3: Disparo de Lembretes na Inicialização (Startup Reminders)

Criou-se um evento para o dia seguinte, com um lembrete de 1 dia antes. Fechou-se o programa, abrindo-o novamente em seguida. Antes de mostrar o calendário, um pop-up foi aberto com o lembrete, conforme esperado.



## 6. Procedimentos de Compilação e Execução

Para assegurar que o avaliador consiga implantar, traduzir e depurar a aplicação sem o uso de empacotadores terceiros, o fluxo base apoia-se estritamente na chamada manual do utilitário padrão do Java SDK pelo terminal de linha de comando:

### Passo 1: Instalação dos Requisitos

Certifique-se de possuir o Kit de Desenvolvimento Java (JDK) instalado na máquina (versão mínima recomendada: JDK 8 ou superior). Verifique digitando em seu prompt de comando:

```
java -version
```

### Passo 2: Organização dos Arquivos

Garanta que todos os arquivos fontes gerados estejam localizados em um mesmo nível de diretório (pasta única de entrega), sem subpastas estruturadas.

### Passo 3: Compilação via Linha de Comando (Bash / Terminal)

Abra o terminal interno do sistema operativo posicionado nesta pasta. Execute a ferramenta de compilação em lote para traduzir o código fonte em bytecodes intermediários executáveis (gerando os respectivos arquivos .class):

```
javac *.java
```

### Passo 4: Execução do Sistema

Invoke a Máquina Virtual Java chamando a classe de interface primária que hospeda a lógica do ponto de entrada do programa (método main):

## 7. Desafios Encontrados

Um dos maiores desafios encontrados pelo grupo no desenvolvimento deste projeto foi coordenar a API de datas nativa do Java (`java.time`) com os índices da grade de botões do calendário visual. Como o primeiro dia de cada mês pode começar em qualquer dia da semana (como uma quarta ou sexta-feira), foi preciso criar uma lógica matemática precisa de deslocamento (*offset*) para que o número "1" fosse desenhado no botão correto da tela do Swing, empurrando os dias vazios para trás de forma dinâmica. Outro obstáculo marcante foi garantir que a persistência de dados no arquivo de texto ocorresse de forma síncrona aos comandos da interface. Foi necessário estruturar cuidadosamente as rotinas de leitura e escrita com tratamento de exceções robusto, assegurando que o encerramento inesperado ou a ausência do arquivo `events_data.txt` na primeira execução não provocassem falhas críticas ou o travamento geral do aplicativo.

## 8. Comentários Finais

O desenvolvimento deste projeto permitiu colocar em prática conceitos importantes de encapsulamento e manipulação de arquivos (I/O) utilizando Java Swing. A versão adotada priorizou uma estrutura mais simples e objetiva, eliminando elementos visuais que não agregavam valor ao funcionamento da aplicação. Com isso, o sistema ficou mais fácil de compreender, testar e manter, além de seguir os princípios e práticas trabalhados ao longo das atividades da disciplina.